together with different combining rules. Many vendors offer implementations of the XACML reference architecture in some form, although often without the standard XML policy language, providing policy agents or gateways and PDP services that you can install into your environment to add ABAC access control decisions to existing services and APIs.

8.3.5 Best practices for ABAC

Although ABAC provides an extremely flexible basis for access control, its flexibility can also be a drawback. It's easy to develop overly complex rules, becoming hard to determine exactly who has access to what. I have heard of deployments with many thousands of policy rules. Small changes to rules can have dramatic impacts, and it can be hard to predict how rules will combine. As an example, I once worked on a system that implemented ABAC rules in the form of XPath expressions that were applied to incoming XML messages; if a message matched any rule it was rejected.

It turned out that a small change to the document structure made by another team caused many of the rules to no longer match, which allowed invalid requests to be processed for several weeks before somebody noticed. It would've been nice to be able to automatically tell when these XPath expressions could no longer match any messages, but due to the flexibility of XPath this turns out to be impossible to determine automatically in general, and all our tests continued using the old format. This anecdote shows the potential downside of flexible policy evaluation